Assignment 1:

Name: Arik Gershman

Part A: Image Colorization (55 Points)

Overview

The goal of Part A is to reconstruct a full-color image from the digitized Prokudin-Gorskii glass plate photographs, which consist of three grayscale images captured through red, green, and blue filters. These three channel images are vertically stacked in a single image and are slightly misaligned due to camera movement and subject motion during exposure.

To produce a visually coherent color image, the three color channels must first be extracted and then precisely aligned with one another before being combined into a single RGB image. The primary challenge lies in accurately aligning the channels so as to minimize visual artifacts. This alignment is performed by searching over possible displacements and selecting the one that maximizes a similarity metric between channels, such as Sum of Squared Differences (SSD) or Normalized Cross-Correlation (NCC). Once the optimal alignment is found, the channels are stacked to form the final color image.

NOTE:

1. Reading this for more background information and samples.
2. You can use any image processing libraries of your choice such as skimage or cv2; in python.

You are required to provide **THREE colorized image results**. Choose ONE of the results, and use it to provide answers in the write-up.

Data

WARNING: Colab deletes all files everytime runtime is disconnected. Make sure to re-download the inputs when it happens.

In [ ]:
```
# Download Data -- run this cell only one time per runtime
!gdown 1KTDxPAkQam29YKtoX5dKPnLKpUOWCanC
!unzip "/content/hybrid_pyramid_input.zip" -d "/content/"

!gdown 19_UNxJ6zAH5-4-ljo-OvnuLKWhtkaChO
!unzip "/content/parta_data.zip" -d "/content/parta_data"
```

```
Downloading...
From: https://drive.google.com/uc?id=1KTDxPAkQam29YKtoX5dKPnLKpUOWCanC
To: /content/hybrid_pyramid_input.zip

  0% 0.00/2.19M [00:00<?, ?B/s]
100% 2.19M/2.19M [00:00<00:00, 70.3MB/s]
Archive:  /content/hybrid_pyramid_input.zip
   creating: /content/data/
  inflating: /content/data/Afghan_girl_before.jpg
  inflating: /content/data/motorcycle.bmp
  inflating: /content/data/cat.bmp
  inflating: /content/data/makeup_before.jpg
  inflating: /content/data/fish.bmp
  inflating: /content/data/bicycle.bmp
  inflating: /content/data/makeup_after.jpg
  inflating: /content/data/plane.bmp
  inflating: /content/data/marilyn.bmp
  inflating: /content/data/dog.bmp
  inflating: /content/data/Afghan_girl_after.jpg
  inflating: /content/data/submarine.bmp
  inflating: /content/data/bird.bmp
  inflating: /content/data/einstein.bmp
Downloading...
From: https://drive.google.com/uc?id=19_UNxJ6zAH5-4-ljo-OvnuLKWhtkaChO
To: /content/parta_data.zip
100% 390k/390k [00:00<00:00, 79.7MB/s]
Archive:  /content/parta_data.zip
  inflating: /content/parta_data/00125v.jpg
  inflating: /content/parta_data/00149v.jpg
  inflating: /content/parta_data/00153v.jpg
  inflating: /content/parta_data/00351v.jpg
  inflating: /content/parta_data/00398v.jpg
  inflating: /content/parta_data/01112v.jpg
```

In [ ]:
```
import os
print(os.listdir('/content/data'))
```

```
['cat.bmp', 'Afghan_girl_before.jpg', 'Afghan_girl_after.jpg', 'makeup_before.jpg', 'bird.bmp', 'bicycle.bmp', 'submarine.bmp', 'motorcycle.bmp', 'makeup_after.jpg', 'fish.bmp', 'marilyn.bmp', 'einstein.bmp', 'dog.bmp', 'plane.bmp']
```

A.0 Know your data (20 pts)

Read and show images **(10 pts)**

Let's start with some helper functions. Implement helper functions below so you can read and visualize images.

In [ ]:
```
# Import necessary packages here
import numpy as np
from PIL import Image

# Helper Functions
def read_image_pil(image_path: str) -> Image:
    """
    :param image_path: path to the image
    :return: representation of the image
    """
    # TODO: YOUR CODE HERE (10 pts)
    # Read an image using PIL.Image.open()
    image = Image.open(image_path)
    return image
```

In [ ]:
```
# Check if you can read and show an image. You are free to change path.
image_path = '/content/data/cat.bmp'
image_pil = read_image_pil(image_path)
# PIL image can be directly shown by just calling it.
image_pil
```

Out[ ]:

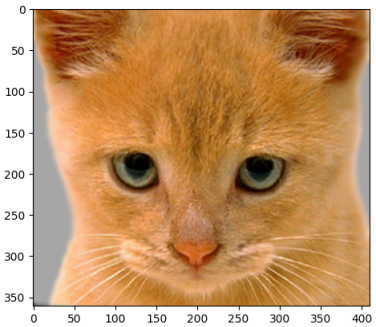
We can do conversions between PIL image format and numpy array. Try commands below.

In [ ]:
```
print(image_pil)  # show the data type of an PIL image
image_np = np.array(image_pil) # PIL image -> numpy array
print('Image shape: ', image_np.shape) # show the shape
print('Data type: ', image_np.dtype) # show the data type
print(image_np) # show exact pixel values
image_pil_new = Image.fromarray(image_np) # numpy array -> PIL image
print(image_pil_new)
```

```
<PIL.BmpImagePlugin.BmpImageFile image mode=RGB size=410x361 at 0x7D76369799A0>
Image shape:  (361, 410, 3)
Data type:  uint8
[[[207 171 121]
  [197 152  96]
  [185 132  67]
  ...
  [208 174 144]
  [216 187 163]
  [218 193 174]]

 [[216 181 136]
  [204 162 109]
  [191 137  79]
  ...
  [208 174 144]
  [215 188 162]
  [219 194 172]]

 [[224 189 148]
  [212 168 120]
  [195 144  91]
  ...
  [208 175 145]
  [216 188 162]
  [221 195 171]]

 ...

 [[ 49  50  53]
  [ 97  98 101]
  [118 119 122]
  ...
  [207 150  88]
  [208 149  87]
  [209 153  89]]

 [[ 39  40  43]
  [ 81  83  85]
  [ 99 103 107]
  ...
  [204 146  85]
  [205 149  87]
  [210 154  92]]

 [[ 36  39  41]
  [ 72  75  76]
  [ 91  94  98]
  ...
  [205 150  87]
  [207 154  91]
  [212 159  96]]]
<PIL.Image.Image image mode=RGB size=410x361 at 0x7D763692CFB0>
```

There are also other ways to show an image. For example, you can also use `matplotlib.pyplot`. Run the code below and get yourself familiar with it.

In [ ]:
```
import matplotlib.pyplot as plt

plt.imshow(image_np)
plt.show()
```



Write-up (10 pts)

1. What is the difference between a `PIL` image and an image as a `numpy.array`? **(5 pt)**
2. What is the data type of an image by default it is directly converted from a `PIL` image? What is the value range? **(5 pts)**

Include your write-up here

1. A `PIL` image is an object, this one was a Bmp Image File, a bitmap representation of the image that separates the original image into different channels. It is in RGB mode, 410 pixels wide, and 361 pixels tall. An image as a `numpy.array` is like a multidimensional matrix that encodes the pixel values of the image. It is structured as a list of 361 lists of 410 lists of 3 values each. Each one of the 361 lists represents a row of pixels, containing 410 lists of 3 values each because each image is 410 pixels wide and is separated into 3 image channels, Red, Green, and Blue.
2. The type of the image by default after conversion is a uint8. The values range from 0 to 255, representing intensity.

A.1 Implementation of Image Colorization (35 pts)

Data type **(5 pts)**

In practice, we need to convert the data type into `float` so that we can apply mathmatical operations. We first write `read_image` function below. Let's use `float64` in this assignment.

In [ ]:
```
def read_image(image_path: str) -> np.ndarray:
    """
    :param image_path: path to the image
    :return: floating representation of the image. Use np.float64.
    """
    # TODO: YOUR CODE HERE (10 pts)
    image_pil = Image.open(image_path)
    image_np = np.array(image_pil)
    image_np_float = image_np.astype(np.float64)
    image = image_np_float / 255.0
    return image
```

Let's check if you are doing it right. The value range should be in [0.0, 1.0].

In [ ]:
```
image_path = '/content/parta_data/00125v.jpg'
img = read_image(image_path)
print(img.shape)
print(img.dtype)
print('value range: [{}, {}]'.format(img.min(), img.max()))
```

```
(1024, 400)
float64
value range: [0.0, 1.0]
```

Helper Functions

In [ ]:
```
def read_and_split_image(image_path: str) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    NO NEED TO CHANGE THIS FUNCTION.
    Reads an image and splits it into its RGB channels.
    :param image_path: path to the image
    :return: R, G, B data
    """
    # read in the image
    im = read_image(image_path)
    # compute the height of each part (just 1/3 of total)
    height = np.floor(im.shape[0] / 3.0).astype(int)

    # separate color channels
    b = im[:height]
    g = im[height:2 * height]
    r = im[2 * height:3 * height]

    return r, g, b

def center_crop(img: np.ndarray, frac: float = 0.5) -> np.ndarray:
    """
    NO NEED TO CHANGE THIS FUNCTION.
    Return the centered crop of an image. frac=0.5 keep the central 50% in both height and width.
    :param img: image array
    :return: cropped image
    """
    h, w = img.shape
    ch, cw = int(h * frac), int(w * frac)
    top = (h - ch) // 2
    left = (w - cw) // 2
    return img[top:top+ch, left:left+cw]
```

Normalized Cross-Correlation (NCC) (5 pts)

NCC measures the similarity between two images while being robust to differences in brightness and contrast. Since the Prokudin-Gorskii color channels were captured using different filters and exposure conditions, raw pixel differences can be misleading. NCC addresses this by normalizing both images to zero mean and unit variance before computing their similarity.

Given two image patches A and B, NCC is computed as:

$$NCC(A, B) = \frac{1}{N} \sum (\frac{A - \mu_A}{\sigma_a})(\frac{B - \mu_B}{\sigma_B})$$

In [ ]:
```
def normalized_cross_correlation(img1: np.ndarray, img2: np.ndarray) -> float:
    """
    Compute the normalized cross-correlation (NCC) between two arrays.
    :param img1, img2: image array
    :return score
    """
    # TODO: YOUR CODE HERE (5 pts)
    # Step 1: get the norm of each image
    img1_norm = (img1 - np.mean(img1)) / np.std(img1)
    img2_norm = (img2 - np.mean(img2)) / np.std(img2)

    # Step 1: compute ncc score from two normalized images
    ncc = np.dot(img1_norm.flatten(), img2_norm.flatten()) / len(img1_norm.flatten())

    return ncc
```

In [ ]:
```
# Sanity check

img = np.random.randn(100, 100)
score = normalized_cross_correlation(img, img)
print(score)
assert score > 0.99, score
```

```
0.9999999999999998
```

Image Alignment (5 pts)

This function aligns one image to a reference image by exhaustively searching over a fixed window of possible shifts. For each candidate displacement, the moving image is shifted, and NCC is computed over the center crop. The displacement that yields the highest NCC score is selected.

In [ ]:
```
def align_images(mov: np.ndarray, ref: np.ndarray, max_shift: int = 32) -> tuple[int, int]:
    """
    Align mov to ref by searching over a window of shifts and maximizing NCC.

    :param ref: reference image (e.g., blue channel)
    :param mov: moving image (e.g., red or green channel)
    :param max_shift: maximum pixel displacement in each direction
    :return: (dx, dy) shift that best aligns mov to ref
    """

    best_dx, best_dy = 0, 0
    best_score = -np.inf

    ref_c = center_crop(ref)

    # TODO: YOUR CODE HERE (5 pts)
    for dx in range(-max_shift, max_shift + 1):
        for dy in range(-max_shift, max_shift + 1):
            # Shift mov by (dx, dy), hint: use np.roll
            shifted_mov_x = np.roll(mov, dx, axis=1) # Shift mov by dx in x dim
            shifted_mov = np.roll(shifted_mov_x, dy, axis=0) # Shift shifted_mov_x by dy in y dim

            # Compute NCC on the centered image to avoid edge effects
            shifted_c = center_crop(shifted_mov) # get the centered shifted_mov
            score = normalized_cross_correlation(ref_c, shifted_c)

            # Update best shift
            if score > best_score:
              best_score = score
              best_dx = dx
              best_dy = dy

    return best_dx, best_dy
```

In [ ]:
```
# Sanity check
np.random.seed(0)
ref = np.random.randn(120, 160)
true_dx, true_dy = 8, -5
mov = np.roll(ref, true_dx, axis=1)
mov = np.roll(mov, true_dy, axis=0)

dx, dy = align_images(ref, mov, max_shift=15)
assert (dx, dy) == (true_dx, true_dy), (dx, dy)
```

Image Colorization (10 pts)

It's time to officially implement Image Colorization.

Let's wrap up all the stuff in a single function `image_colorization`. The key idea is to choose one channel as the reference, then align the remaining two channels to this reference using the alignment function implemented earlier. In the provided hint code, the blue channel is used as the reference, but you are encouraged to experiment with using the red or green channel instead and compare the visual results.

After computing the optimal shifts, apply them to align the channels. Finally, stack the aligned red, green, and blue channels together to form the reconstructed RGB color image.

In [ ]:
```
def image_colorization(image_path: str):
    """
    :param image_path: path to the image
    :return: Colorized image
    """
    r, g, b = read_and_split_image(image_path)

    # TODO: YOUR CODE HERE (5 pts)
    # Step 1: get the best shifts
    dx_r, dy_r = align_images(r, b) # shift red channel take blue channel as reference
    dx_g, dy_g = align_images(g, b) # shift green channel take blue channel as reference

    # Step 2: apply the shifts, hint: use np.roll
    r = np.roll(r, dx_r, axis=1)
    r = np.roll(r, dy_r, axis=0)
    g = np.roll(g, dx_g, axis=1)
    g = np.roll(g, dy_g, axis=0)

    # Step 3: create a color image, hint: use np.stack
    color_img = np.stack((r,g,b), axis=-1)

    return color_img
```

Let's see if we are doing it right. You should see the colorized image like This.

In [ ]:
```
colorized_img = image_colorization('/content/parta_data/00125v.jpg')

# Display the image
plt.imshow(colorized_img)
plt.axis('off')
plt.show()
```

Write-up (10 pts)

1. Run `image_colorization` on 3 different images. Report your results below. (5 pts)
2. Briefly explain how this works, using your favorite results as illustrations. (5 pt)

Include your write-up here

1.

2. The idea is taking 3 images that represent the intensity of each color (red, blue, and green) for each pixel, and stack them together to make one colorful image. After reading and splitting the image into the 3 RGB channels and before stacking them to make a colorful image, we must ensure that the images are suited to be stacked, since they may be misaligned. We align the image by looking for the best possible shifts that maximize the NCC of the shifted, centered image. We go through each possible combination of dx and dy, shift the image by those respective amount, center crop it, and calculate the Normalized Cross-Correlation (which is a robust way to measure similarity between two images). We find the highest NCC score and return the shifts (dx and dy) that caused that score. After that, we simply apply the shifts and stack the images on top of each other to produce the colorful image. My favorite result is the image with the river (first image above). For that image, the top 2 images in the original file (especially the top one) are very bright where the sky is. That is beacuse the sky is a mix of blue and green, whereas the bottom image is much darker at the sky since it's not as red. Additionally, the area where the grass is is much more intense in the 2nd image than in the other two beacuse, of course, the grass is green. The buildings on the left are pretty intense in all 3 images, and that is because white is composed of high intensity red, blue, and green pixels. Therefore, after using the best shifts found using the NCC and stacking the 3 channels, a colorful image is created where the sky is light blue/turquoise, the grass is green, and the buildings are white.

Part B: Hybrid Image (45 Points)

Overview

A hybrid image is the sum of a *low-pass filtered* version of the one image and a *high-pass filtered* version of a second image. There is a free parameter, which can be tuned for each image pair, which controls how much high frequency to remove from the first image and how much low frequency to leave in the second image. This is called the "cutoff-frequency". In the paper it is suggested to use two cutoff frequencies (one tuned for each image) and you are free to try that, as well. In the starter code, the cutoff frequency is controlled by changing the standard deviation of the Gausian filter used in constructing the hybrid images. This is the sample example.

NOTE:

1. Reading this will help in understanding Part B.
2. You can use any image processing libraries of your choice such as skimage or cv2; in python.

We provided 7 pairs of aligned images. The alignment is important because it affects the perceptual grouping (read the paper for details). We encourage you to create additional examples (e.g. change of expression, morph between different objects, change over time, etc.).

You are required to provide **THREE hybrid image results**. Choose ONE of the results, and use it to provide answers in the write-up.

B.1 Implementation of Hybrid Image (55 pts)

Now let's start to implement hybrid image. We will start again with some helper functions.

Helper Functions

In [ ]:
```
def vis_hybrid_image(hybrid_image) -> np.ndarray:
  """
  NO NEED TO CHANGE THIS FUNCTION.
  Visualize a hybrid image by progressively downsampling the image and
  concatenating all of the images together.
  :param hybrid_image:
  :return:
  """
  scales = 5
  scale_factor = 0.5
  padding = 5
  original_height = hybrid_image.shape[0]
  num_colors = hybrid_image.shape[2] # counting how many color channels the input has
  output = hybrid_image
  cur_image = hybrid_image

  for i in range(2, scales):
      # add padding
      output = np.concatenate((output, np.ones((original_height, padding, num_colors), dtype=int)), axis=1)
      # dowsample image;
      width = int(cur_image.shape[1] * scale_factor)
      height = int(cur_image.shape[0] * scale_factor)
      dim = (width, height)
      cur_image = cv2.resize(cur_image, dim, interpolation = cv2.INTER_LINEAR)
      # pad the top and append to the output
      tmp = np.concatenate((np.ones((original_height-cur_image.shape[0], cur_image.shape[1], num_colors)), cur_image), axis=0)
      output = np.concatenate((output, tmp), axis=1)

  output = (output * 255).astype(np.uint8)
  return output
```

2D Gaussian filters **(15 pts)**

We will implement a 2D Gaussian filter function in `gaussian_2D_filter`. **(12 pts)**

Recall from our lectures that a 2D Gaussian filter is a matrix/tensor filled with values, represented as:

$$G(x, y) = \frac{1}{2\pi\sigma^2}\exp^{-\frac{x^2+y^2}{2\sigma^2}},$$

where x and y denote the distances from the origin along the horizontal and vertical axes, respectively, and σ represents the standard deviation, which controls the extent of the blur or "cut-off frequency".

Hint 1: You probably want to get those coordinates (x, y) so that you can compute values at each pixel. In practice, it is easier to get the coordinates of pixels with `np.meshgrid`. Check out an example here (See Examples).

Hint 2: In `numpy`, most of the functions are element-wise, meaning they apply the same operation to every element in an array, e.g., `np.exp`.

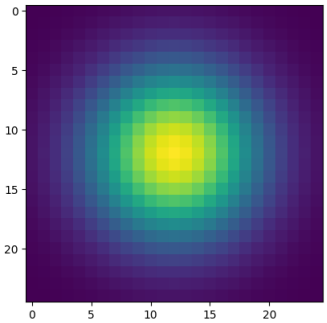
To apply a filter to an image, implement `imgfilter`. You can use `scipy.signal.convolve2d` (check out an example here) .**(3 pts)**

In [ ]:
```
import cv2
from scipy.signal import convolve2d

def gaussian_2D_filter(size: tuple[int, int], sigma: int) -> np.ndarray:
    """
    :param size: tuple (width, height) that decides the filter size
    :param sigma: standard deviation, hyperparameter to control the variance of the filter
    :return: 2D gaussian filter with the desired size, and variance scaled by cutoff_frequency
    Hint: make sure the filter sums up to one
    Do NOT use scipy's API to get the filter. Please just use numpy to implement the Gaussian equation.
    """
    # TODO: YOUR CODE HERE (5 pts)
    # Step 1: Make a mesh grid using `np.meshgrid`. (Why?)
    x_range = np.arange(size[0])
    x = x_range - np.mean(x_range)
    y_range = np.arange(size[1])
    y = y_range - np.mean(y_range)

    xv, yv = np.meshgrid(x,y)

    # Step 2: Compute Gaussian function with `np.exp` and the mesh grid.
    filter = (1.0/(2.0*np.pi*(sigma**2))) * np.exp(-(xv**2 + yv**2)/(2*(sigma**2)))
```

```
    # Step 3: Normalize so that sum equals 1
    filter = filter / np.sum(filter)

    return filter

def imgfilter(image: np.ndarray, filter: np.ndarray, conv_mode='same', boundary='symm') -> np.ndarray:
    """
    Apply a 2D filter to an image using convolution. Supports both grayscale and RGB images.

    :param image: input image (grayscale or RGB) to apply the filter on
    :param filter: the filter to apply on the image
    :param conv_mode: 'same' or 'valid'
    :param boundary: 'symm', 'reflect', or 'wrap'
    :return: apply the filter by convolving
    Do NOT use for loops. See how to convolve with scipy.signal.convolve2d.
    """
    # TODO: YOUR CODE HERE (5 pts)
    # Step 1: check if it is a grayscale image or a RGB image.
    if len(image.shape) == 3:
      image = np.transpose(image)
      channels = []
      for i in range(3):
        channels.append(convolve2d(image[i], filter, mode=conv_mode, boundary=boundary))
      output = np.transpose(np.stack(channels))

    # Step 2: apply the filter to it
    else:
      output = convolve2d(image, filter, mode=conv_mode, boundary=boundary)

    return output
```

In [ ]:
```
# Sanity check
filter_example = gaussian_2D_filter((25, 25), sigma=5.0)
print(filter_example.shape)
print('Sum of the filter should be 1: ', filter_example.sum())

# Visualize it
plt.imshow(filter_example)
plt.show()
```

```
(25, 25)
Sum of the filter should be 1:  1.0
```



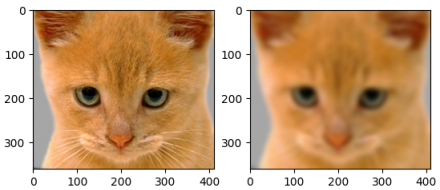
Visualize FFT (10 pts)

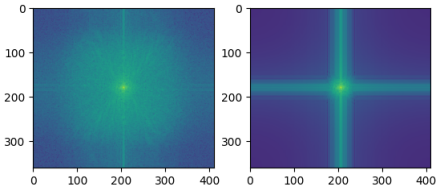
After applying a Gaussian filter, high-frequency component is reduced. A way to check one's frequency domain is to use Fast Fourier Transform (FFT). Let's now implement `log_mag_FFT` function below.

In [ ]:
```
def log_mag_FFT(image: np.ndarray) -> np.ndarray:
    """
    :param image: float matrix representation of the image
    :return: log of the magnitude of the FFT of the image
    HINT: You may use np.log(np.abs(np.fft.fftshift(np.fft.fft2(image)))) to achieve it.
    NOTE1: numpy fft2 would require you to convert the image to grayscale for it to work properly.
    NOTE2: To make grayscale, you may use either `grey = R*0.3 + G*0.59 + B*0.11` or `cv2.cvtColor`.
    """
    # TODO: YOUR CODE HERE
    image = image.astype(np.float32)
    # Step 1: convert an image to grayscale if needed.
    if len(image.shape) == 3:
      image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

    # Step 2: compute log magnitute.
    output = np.log(np.abs(np.fft.fftshift(np.fft.fft2(image))))

    return output
```

Let's see how we have done so far. Below we read an image, create and apply a Gaussian filter to it, and visualize the log magnitute of the FFT.

In [ ]:
```
# Read an image
img_path = '/content/data/cat.bmp'
img = read_image(img_path)

# make a Gaussian filter. Feel free to modify parameters.
filter_size = 25
sigma = 5.0
gaussian_filter = gaussian_2D_filter((filter_size, filter_size), sigma)

# apply the filter
img_low_freq = imgfilter(img, gaussian_filter)

# visualize images (before and after, side by side)
plt.subplot(1, 2, 1)
plt.imshow(img)
plt.subplot(1, 2, 2)
plt.imshow(img_low_freq)
plt.show()

# visualize the FFT (before and after, side by side)
plt.subplot(1, 2, 1)
plt.imshow(log_mag_FFT(img))
plt.subplot(1, 2, 2)
plt.imshow(log_mag_FFT(img_low_freq))
plt.show()
```





Hybrid image (10 pts)

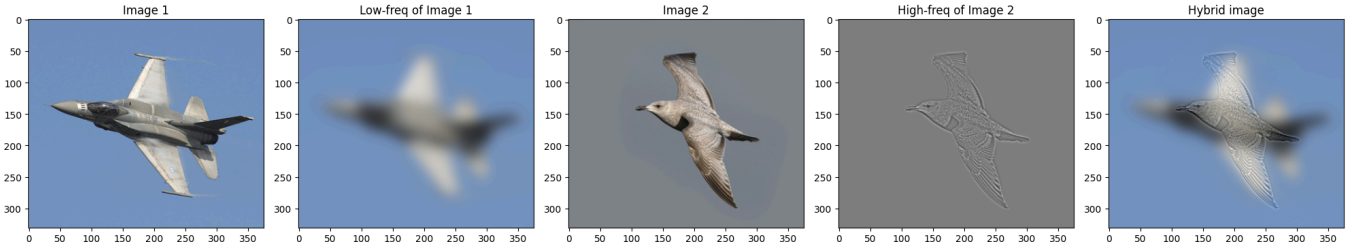
It's time to officially implement Hybrid Image. You're expected to compose two images together.

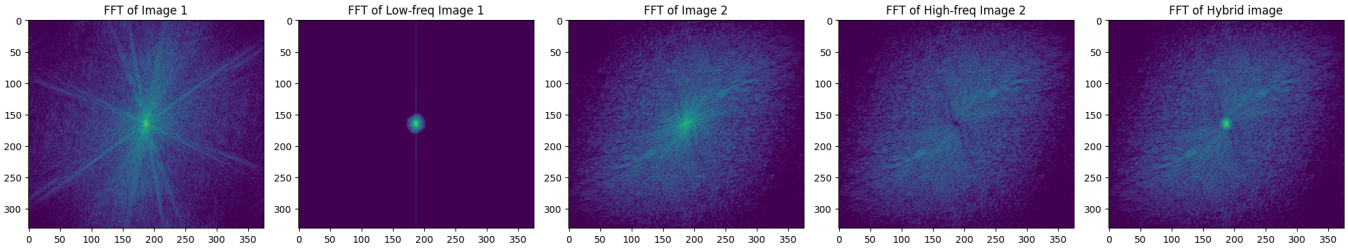
Let's wrap up all the stuff in a single function `hybrid_image`.

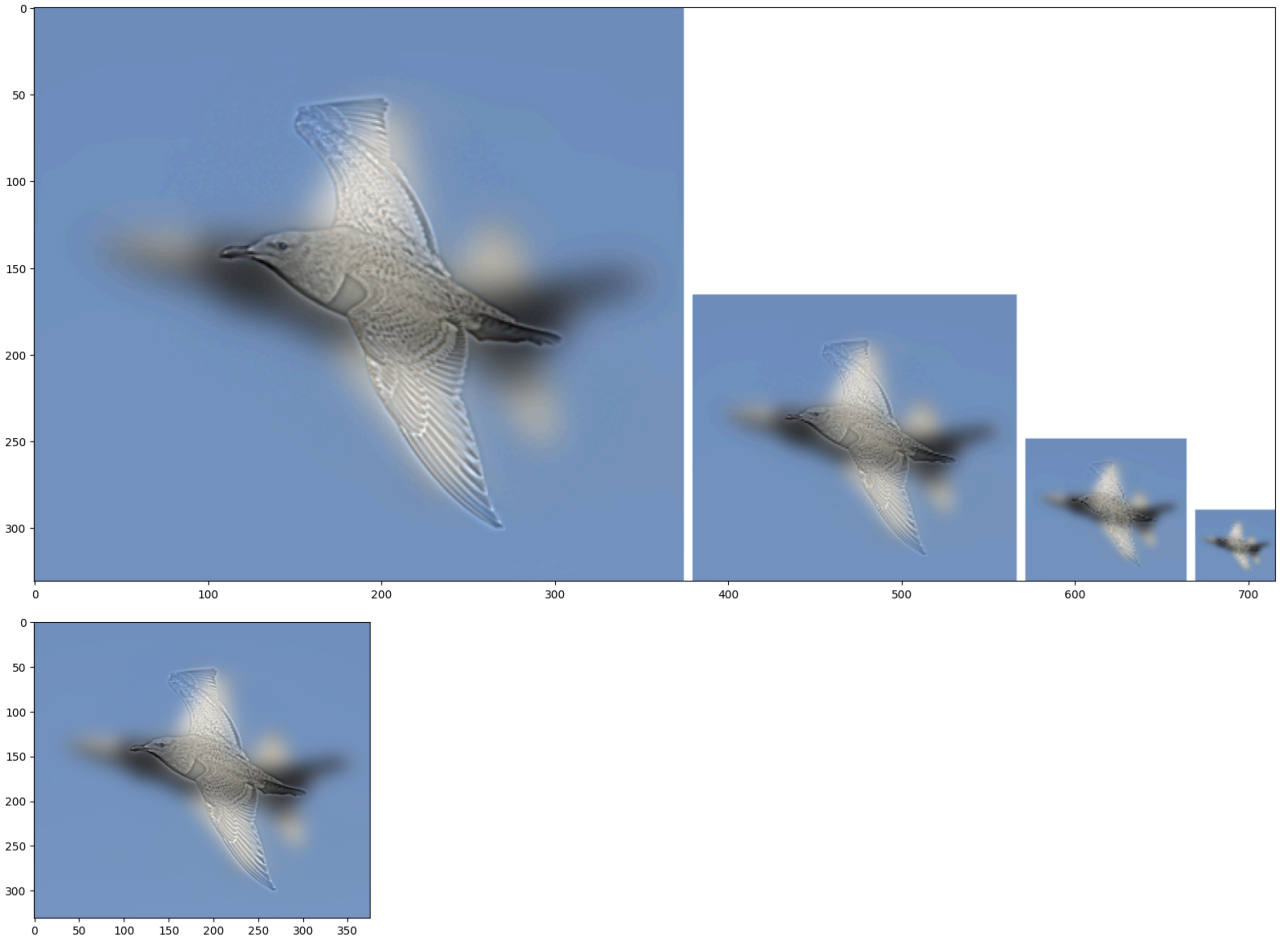
In [ ]:
```
from re import I
# Finish hybrid_image.
def hybrid_image(image1: np.ndarray, image2: np.ndarray, sigma_low_freq: int, sigma_high_freq: int, if_visual=False):
    """
    :param image1: image of type float64
    :param image2: image of type float64
    :param sigma_low_freq: Standard deviation for the low-pass filter (image1)
    :param sigma_high_freq: Standard deviation for the high-pass filter (image2)
    :return: Hybrid image combining low-frequency and high-frequency content
    """

    # Determine filter size based on sigma (typically 6*sigma covers most of the Gaussian)
    filter_size_low = (int(6 * sigma_low_freq) | 1, int(6 * sigma_low_freq) | 1)
    filter_size_high = (int(6 * sigma_high_freq) | 1, int(6 * sigma_high_freq) | 1)

    ###### TODO START #######
    # TODO 1: Finish hybrid image.
    # Step 1: Create Gaussian filters
    low_pass_filter = gaussian_2D_filter(filter_size_low, sigma_low_freq)
    high_pass_filter = gaussian_2D_filter(filter_size_high, sigma_high_freq)

    # Step 2: Apply low-pass filter to image1
    low_frequencies = imgfilter(image1, low_pass_filter)

    # Step 3: Apply high-pass filter to image2 and subtract from original to get high frequencies
    low_from_image2 = imgfilter(image2, high_pass_filter)
    high_frequencies = image2 - low_from_image2

    # Step 4: Combine the two components
    hybrid = low_frequencies + high_frequencies

    # Step 5: clip output value range. See np.clip().
    output = np.clip(hybrid, 0.0, 1.0)

    ###### TODO END ######

    if if_visual:
      # Visualization code. NO NEED TO CHANGE.
      # visualize two input images, sigma_low_freq, sigma_high_freq, and the hybrid image.
      fig, axs = plt.subplots(2, 5, figsize=(20, 10))
      axs[0, 0].imshow(image1)
      axs[0, 1].imshow(low_frequencies)
      axs[0, 2].imshow(image2)
      axs[0, 3].imshow(high_frequencies + 0.5)
      axs[0, 4].imshow(output)

      # visualize log magnitude of Fourier Transform of the above.
      axs[1, 0].imshow(log_mag_FFT(image1), vmin=0, vmax=10)
      axs[1, 1].imshow(log_mag_FFT(low_frequencies), vmin=0, vmax=10)
      axs[1, 2].imshow(log_mag_FFT(image2), vmin=0, vmax=10)
      axs[1, 3].imshow(log_mag_FFT(high_frequencies), vmin=0, vmax=10)
      axs[1, 4].imshow(log_mag_FFT(output), vmin=0, vmax=10)

      axs[0, 0].set_title('Image 1')
      axs[0, 1].set_title('Low-freq of Image 1')
      axs[0, 2].set_title('Image 2')
      axs[0, 3].set_title('High-freq of Image 2')
      axs[0, 4].set_title('Hybrid image')
      axs[1, 0].set_title('FFT of Image 1')
      axs[1, 1].set_title('FFT of Low-freq Image 1')
      axs[1, 2].set_title('FFT of Image 2')
      axs[1, 3].set_title('FFT of High-freq Image 2')
      axs[1, 4].set_title('FFT of Hybrid image')
      #plt.subplots_adjust(hspace=0)
      fig.tight_layout()
      plt.show()

      # visualize hybrid_image_scale using helper function vis_hybrid_image.
      fig, ax = plt.subplots(figsize=(20, 10))
      ax.imshow(vis_hybrid_image(output))
      plt.show()



    return output  # Ensure valid pixel range for floating point images (0-1)
```

Let's see if we are doing it right. You should see a hybrid image of a dog and a cat.

In [ ]:
```
# read two images
image_path1 = '/content/data/plane.bmp'
image_path2 = '/content/data/bird.bmp'
image1 = read_image(image_path1)
image2 = read_image(image_path2)

# do hybrid image
hybrid_output = hybrid_image(image1, image2, sigma_low_freq=10, sigma_high_freq=2, if_visual=True)

# show output
plt.imshow(hybrid_output)
plt.show()
```







Write-up (10 pts)

1. Run `hybrid_image` on 3 different combinations. Report your results below. (5 pts) NOTE: You should always compose images that have the same shape.
2. What happened to the FFTs? Anything changed? Explain what you have found. (2 pts)
3. Briefly explain how this works, using your favorite results as illustrations. (3 pt)

Include your write-up here

1.

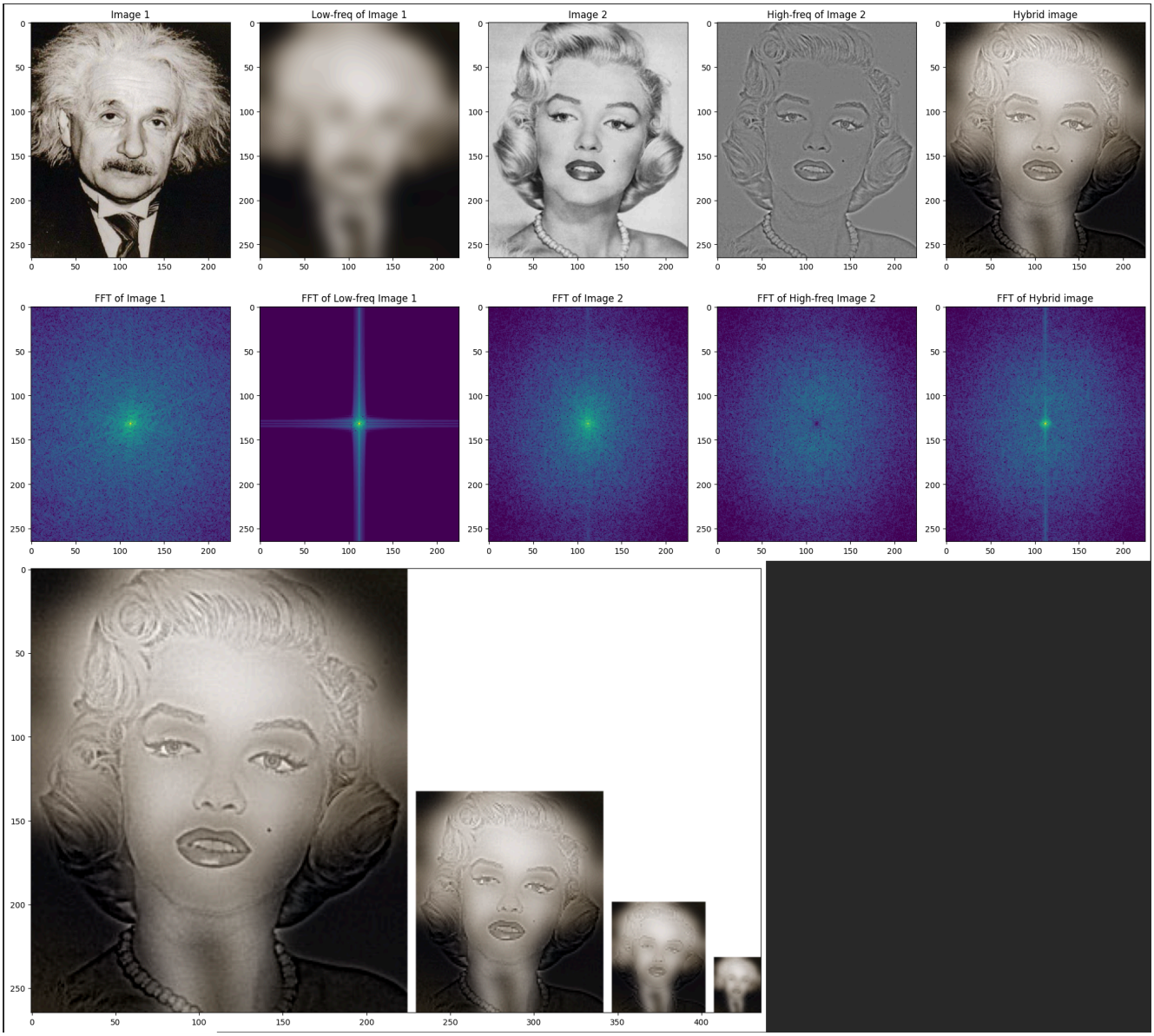


Image 1
Low-freq of Image 1
Image 2
High-freq of Image 2
Hybrid image
FFT of Image 1
FFT of Low-freq Image 1
FFT of Image 2
FFT of High-freq Image 2
FFT of Hybrid image

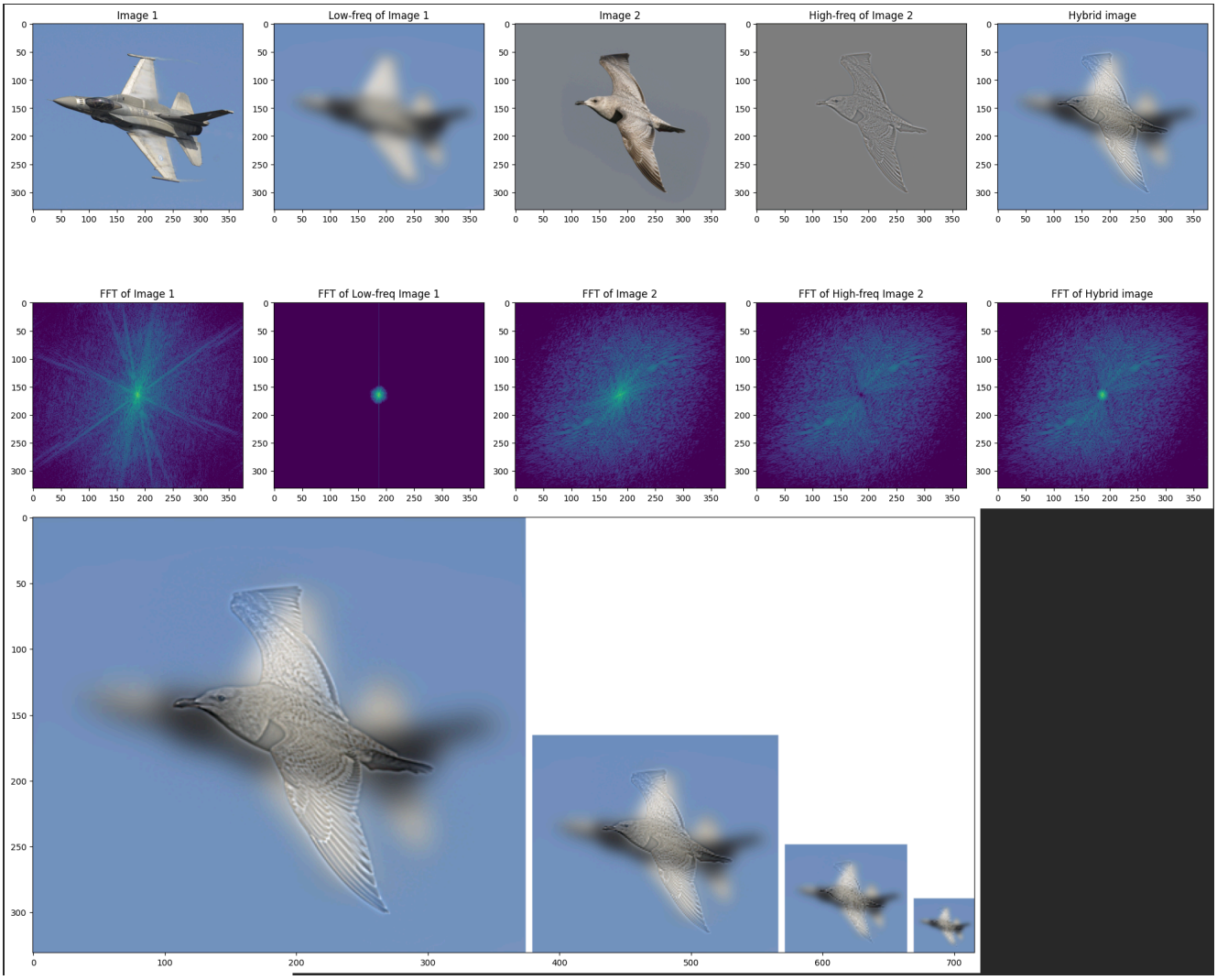
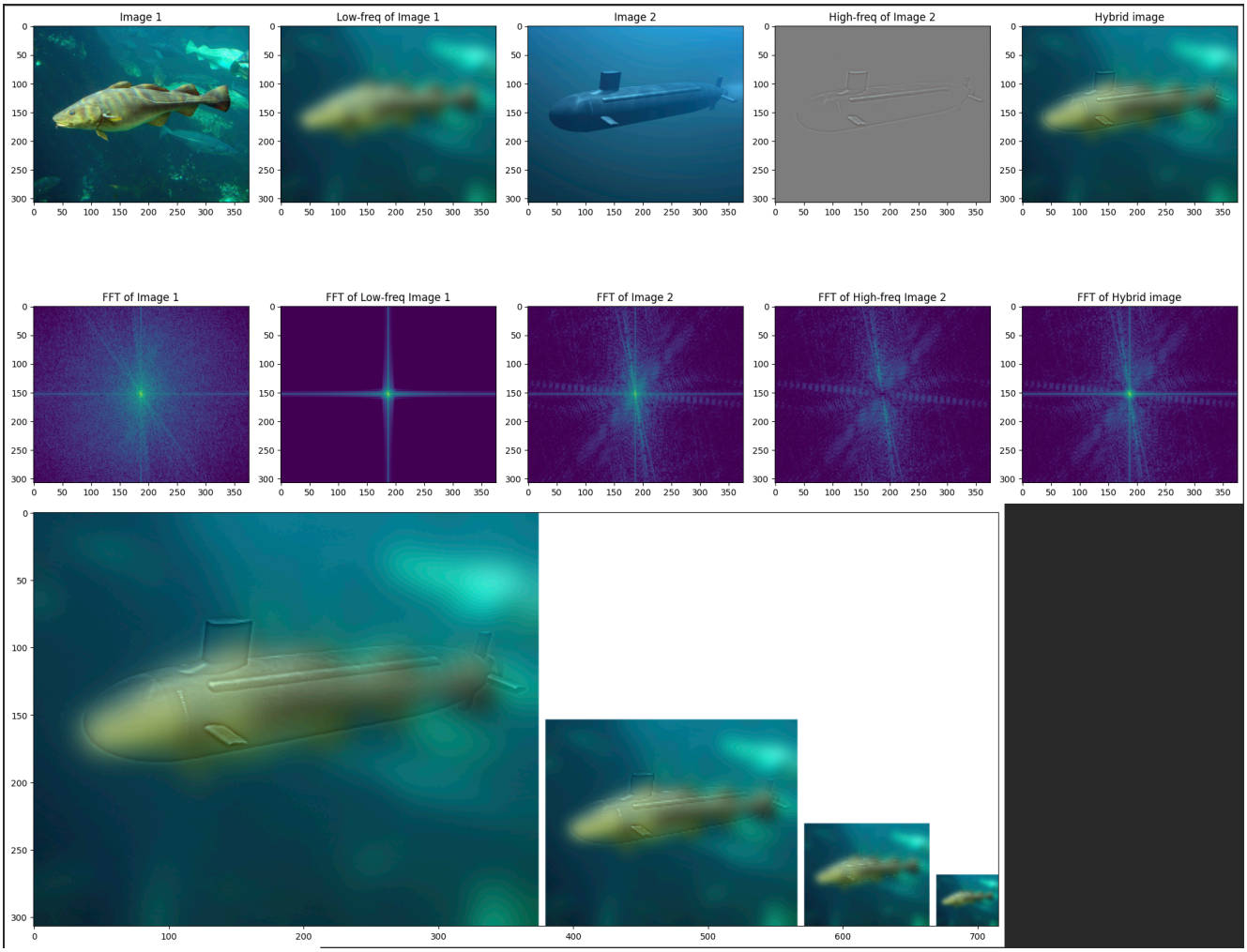


Image 1
Low-freq of Image 1
Image 2
High-freq of Image 2
Hybrid image
FFT of Image 1
FFT of Low-freq Image 1
FFT of Image 2
FFT of High-freq Image 2
FFT of Hybrid image



2. The FFTs of the low-frequency images had reduced high-frequency data in its graph. The low-frequency FFT contained the brightest spots of the FFT of the original image, generally showing a bright spot in the middle, a vertical line through that point, and a horizontal line through that point. The high-frequency FFTs reduced the low-frequency data. They basically contained everything but those bright spots that the low-frequency FFT contained. They were shown as they were missing a dot in the middle and vertical/horizontal lines going through the middle. Together, the final FFT had both low-frequency and high-frequency components that formed the hybrid image.
3. The goal of creating a hybrid image is to combine two images, where if we look from a close distance we see one image and if we look from farther away we see the other image. This is done through extracting low- and high-frequency components of both images and combining them. First, we pick the image we want to be able to see from farther away (low-frequency image) and the image we want to be able to see from closer (high-frequency image). Then, we apply Gaussian filters to each image. Since the Gaussian filter reduces the high-frequency components, the low-frequency image is completed, but in order to get the high-frequency image we must subtract the filtered image from the original image. Finally, we combine the two images to achieve a hybrid image. My favorite result was the bird and plane hybrid image. The low-frequency image of the plane is a more blurred version of the original image, whereas the high-frequency image of the bird sort of depicts the edges and the outline of the bird. Due to that, our brain interprets the blurred image of the plane as noise from close, and we see more of the bird (the high-frequency image). From farther away, our eyes cannot see the outline of the bird, so we see the plane (the low-frequency image).

Save as a PDF file

Make sure you have run Dependecies below. It will take a while. You will find PDF file on your left hand side. If the following code does not work for you, go `File > Print > Save`.

In [ ]:
```
!apt-get install texlive texlive-xetex texlive-latex-extra pandoc
!pip install pypandoc
```

```
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
pandoc is already the newest version (2.9.2.1-3ubuntu2).
pandoc set to manually installed.
The following additional packages will be installed:
  dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
  fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
  libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
  libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
  libsynctex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzzip-0-13
  lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
  ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
  teckit tex-common tex-gyre texlive-base texlive-binaries
  texlive-fonts-recommended texlive-latex-base texlive-latex-recommended
  texlive-pictures texlive-plain-generic tipa xfonts-encodings xfonts-utils
Suggested packages:
  fonts-noto fonts-freefont-otf | fonts-freefont-ttf libavalon-framework-java
  libcommons-logging-java-doc libexcalibur-logkit-java liblog4j1.2-java
  poppler-utils ghostscript fonts-japanese-mincho | fonts-ipafont-mincho
  fonts-japanese-gothic | fonts-ipafont-gothic fonts-arphic-ukai
  fonts-arphic-uming fonts-nanum ri ruby-dev bundler debhelper gv
  | postscript-viewer perl-tk xpdf | pdf-viewer xzdec
  texlive-fonts-recommended-doc texlive-latex-base-doc python3-pygments
  icc-profiles libfile-which-perl libspreadsheet-parseexcel-perl
  texlive-latex-extra-doc texlive-latex-recommended-doc texlive-luatex
  texlive-pstricks dot2tex prerex texlive-pictures-doc vprerex
  default-jre-headless tipa-doc
The following NEW packages will be installed:
  dvisvgm fonts-droid-fallback fonts-lato fonts-lmodern fonts-noto-mono
  fonts-texgyre fonts-urw-base35 libapache-pom-java libcommons-logging-java
  libcommons-parent-java libfontbox-java libgs9 libgs9-common libidn12
  libijs-0.35 libjbig2dec0 libkpathsea6 libpdfbox-java libptexenc1 libruby3.0
  libsynctex2 libteckit0 libtexlua53 libtexluajit2 libwoff1 libzzip-0-13
  lmodern poppler-data preview-latex-style rake ruby ruby-net-telnet
  ruby-rubygems ruby-webrick ruby-xmlrpc ruby3.0 rubygems-integration t1utils
  teckit tex-common tex-gyre texlive texlive-base texlive-binaries
  texlive-fonts-recommended texlive-latex-base texlive-latex-extra
  texlive-latex-recommended texlive-pictures texlive-plain-generic
  texlive-xetex tipa xfonts-encodings xfonts-utils
0 upgraded, 54 newly installed, 0 to remove and 2 not upgraded.
Need to get 182 MB of archives.
After this operation, 571 MB of additional disk space will be used.
Get:1 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-droid-fallback all 1:6.0.1r16-1.1build1 [1,805 kB]
Get:2 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-lato all 2.0-2.1 [2,696 kB]
Get:3 http://archive.ubuntu.com/ubuntu jammy/main amd64 poppler-data all 0.4.11-1 [2,171 kB]
Get:4 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tex-common all 6.17 [33.7 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-urw-base35 all 20200910-1 [6,367 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgs9-common all 9.55.0~dfsg1-0ubuntu5.13 [753 kB]
Get:7 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libidn12 amd64 1.38-4ubuntu1 [60.0 kB]
Get:8 http://archive.ubuntu.com/ubuntu jammy/main amd64 libijs-0.35 amd64 0.35-15build2 [16.5 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy/main amd64 libjbig2dec0 amd64 0.19-3build2 [64.7 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libgs9 amd64 9.55.0~dfsg1-0ubuntu5.13 [5,032 kB]
Ign:11 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libkpathsea6 amd64 2021.20210626.59705-1ubuntu0.2
Get:12 http://archive.ubuntu.com/ubuntu jammy/main amd64 libwoff1 amd64 1.0.2-1build4 [45.2 kB]
Get:13 http://archive.ubuntu.com/ubuntu jammy/universe amd64 dvisvgm amd64 2.13.1-1 [1,221 kB]
Get:14 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-lmodern all 2.004.5-6.1 [4,532 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy/main amd64 fonts-noto-mono all 20201225-1build1 [397 kB]
Err:11 http://security.ubuntu.com/ubuntu jammy-updates/main amd64 libkpathsea6 amd64 2021.20210626.59705-1ubuntu0.2
  404  Not Found [IP: 91.189.92.24 80]
Get:16 http://archive.ubuntu.com/ubuntu jammy/universe amd64 fonts-texgyre all 20180621-3.1 [10.2 MB]
Get:17 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libapache-pom-java all 18-1 [4,720 B]
Get:18 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-parent-java all 43-1 [10.8 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libcommons-logging-java all 1.2-2 [60.3 kB]
Ign:20 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libptexenc1 amd64 2021.20210626.59705-1ubuntu0.2
Get:21 http://archive.ubuntu.com/ubuntu jammy/main amd64 rubygems-integration all 1.18 [5,336 B]
Get:22 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby3.0 amd64 3.0.2-7ubuntu2.11 [50.1 kB]
Get:23 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby-rubygems all 3.3.5-2ubuntu1.2 [228 kB]
Get:24 http://archive.ubuntu.com/ubuntu jammy/main amd64 ruby amd64 1:3.0~exp1 [5,100 B]
Get:25 http://archive.ubuntu.com/ubuntu jammy/main amd64 rake all 13.0.6-2 [61.7 kB]
Err:20 http://security.ubuntu.com/ubuntu jammy-updates/main amd64 libptexenc1 amd64 2021.20210626.59705-1ubuntu0.2
  404  Not Found [IP: 91.189.92.24 80]
Get:26 http://archive.ubuntu.com/ubuntu jammy/main amd64 ruby-net-telnet all 0.1.1-2 [12.6 kB]
Get:27 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby-webrick all 1.7.0-3ubuntu0.2 [52.5 kB]
Get:28 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 ruby-xmlrpc all 0.3.2-1ubuntu0.1 [24.9 kB]
Get:29 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libruby3.0 amd64 3.0.2-7ubuntu2.11 [5,114 kB]
Ign:30 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libsynctex2 amd64 2021.20210626.59705-1ubuntu0.2
Get:31 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libteckit0 amd64 2.5.11+ds1-1 [421 kB]
Ign:32 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtexlua53 amd64 2021.20210626.59705-1ubuntu0.2
Ign:33 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 libtexluajit2 amd64 2021.20210626.59705-1ubuntu0.2
Get:34 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libzzip-0-13 amd64 0.13.72+dfsg.1-1.1 [27.0 kB]
Get:35 http://archive.ubuntu.com/ubuntu jammy/main amd64 xfonts-encodings all 1:1.0.5-0ubuntu2 [578 kB]
Get:36 http://archive.ubuntu.com/ubuntu jammy/main amd64 xfonts-utils amd64 1:7.7+6build2 [94.6 kB]
Get:37 http://archive.ubuntu.com/ubuntu jammy/universe amd64 lmodern all 2.004.5-6.1 [9,471 kB]
Err:30 http://security.ubuntu.com/ubuntu jammy-updates/main amd64 libsynctex2 amd64 2021.20210626.59705-1ubuntu0.2
  404  Not Found [IP: 91.189.92.24 80]
Err:32 http://security.ubuntu.com/ubuntu jammy-updates/main amd64 libtexlua53 amd64 2021.20210626.59705-1ubuntu0.2
  404  Not Found [IP: 91.189.92.24 80]
Err:33 http://security.ubuntu.com/ubuntu jammy-updates/main amd64 libtexluajit2 amd64 2021.20210626.59705-1ubuntu0.2
  404  Not Found [IP: 91.189.92.24 80]
Get:38 http://archive.ubuntu.com/ubuntu jammy/universe amd64 preview-latex-style all 12.2-1ubuntu1 [185 kB]
Get:39 http://archive.ubuntu.com/ubuntu jammy/main amd64 t1utils amd64 1.41-4build2 [61.3 kB]
Get:40 http://archive.ubuntu.com/ubuntu jammy/universe amd64 teckit amd64 2.5.11+ds1-1 [699 kB]
Get:41 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tex-gyre all 20180621-3.1 [6,209 kB]
Ign:42 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 texlive-binaries amd64 2021.20210626.59705-1ubuntu0.2
Get:43 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-base all 2021.20220204-1 [21.0 MB]
Err:42 http://security.ubuntu.com/ubuntu jammy-updates/universe amd64 texlive-binaries amd64 2021.20210626.59705-1ubuntu0.2
  404  Not Found [IP: 91.189.92.24 80]
Get:44 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-fonts-recommended all 2021.20220204-1 [4,972 kB]
Get:45 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-base all 2021.20220204-1 [1,128 kB]
Get:46 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-recommended all 2021.20220204-1 [14.4 MB]
Get:47 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive all 2021.20220204-1 [14.3 kB]
Get:48 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libfontbox-java all 1:1.8.16-2 [207 kB]
Get:49 http://archive.ubuntu.com/ubuntu jammy/universe amd64 libpdfbox-java all 1:1.8.16-2 [5,199 kB]
Get:50 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-pictures all 2021.20220204-1 [8,720 kB]
Get:51 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-latex-extra all 2021.20220204-1 [13.9 MB]
Get:52 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-plain-generic all 2021.20220204-1 [27.5 MB]
Get:53 http://archive.ubuntu.com/ubuntu jammy/universe amd64 tipa all 2:1.3-21 [2,967 kB]
Get:54 http://archive.ubuntu.com/ubuntu jammy/universe amd64 texlive-xetex all 2021.20220204-1 [12.4 MB]
Fetched 171 MB in 11s (16.0 MB/s)
E: Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/t/texlive-bin/libkpathsea6_2021.20210626.59705-1ubuntu0.2_amd64.deb  404  Not Found [IP: 91.189.92.24 80]
E: Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/t/texlive-bin/libptexenc1_2021.20210626.59705-1ubuntu0.2_amd64.deb  404  Not Found [IP: 91.189.92.24 80]
E: Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/t/texlive-bin/libsynctex2_2021.20210626.59705-1ubuntu0.2_amd64.deb  404  Not Found [IP: 91.189.92.24 80]
E: Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/t/texlive-bin/libtexlua53_2021.20210626.59705-1ubuntu0.2_amd64.deb  404  Not Found [IP: 91.189.92.24 80]
E: Failed to fetch http://security.ubuntu.com/ubuntu/pool/main/t/texlive-bin/libtexluajit2_2021.20210626.59705-1ubuntu0.2_amd64.deb  404  Not Found [IP: 91.189.92.24 80]
E: Failed to fetch http://security.ubuntu.com/ubuntu/pool/universe/t/texlive-bin/texlive-binaries_2021.20210626.59705-1ubuntu0.2_amd64.deb  404  Not Found [IP: 91.189.92.24 80]
E: Unable to fetch some archives, maybe run apt-get update or try with --fix-missing?
Collecting pypandoc
  Downloading pypandoc-1.16.2-py3-none-any.whl.metadata (16 kB)
Downloading pypandoc-1.16.2-py3-none-any.whl (19 kB)
Installing collected packages: pypandoc
Successfully installed pypandoc-1.16.2
```

You will be prompted to allow permissions to Google Drive.

In [ ]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

In [ ]:
```
!cp /content/drive/MyDrive/CMSC426/CMSC426_Assignment1_sp26.ipynb ./
!jupyter nbconvert --to html "CMSC426_Assignment1_sp26.ipynb"
```

```
[NbConvertApp] Converting notebook CMSC426_Assignment1_sp26.ipynb to html
[NbConvertApp] WARNING | Alternative text is missing on 9 image(s).
[NbConvertApp] Writing 14473434 bytes to CMSC426_Assignment1_sp26.html
```